Lecture # 12

By: Muhammad Zain Uddin

email: zuddin@iba.edu.pk

Revision

pipe()

Redirecting process I/O

Agenda

Interprocess Communication

- FIFO
- Shared Memory

Attribution

TODAY'S LECTURE IS BASED ON BEEJ'S GUIDE TO INTERPROCESS COMMUNICATION

Question

WHAT IS THE LIMITATION OF USING PIPE?

FIFO (Name Pipe)

Definition:

FIFOs, or named pipes, are similar to pipes but have a name in the filesystem, allowing unrelated processes to communicate.

Characteristics:

Named pipes provide named, bidirectional communication channels between unrelated processes.

Created using the mkfifo() system call or the mkfifo command.

FIFO

A FIFO (“First In, First Out”, pronounced “Fy-Foh”) is sometimes known as a named pipe.

It’s like a pipe, except that it has a name! I

- n this case, the name is that of a file that multiple processes can open() and read and write to.

This latter aspect of FIFOs is designed to let them get around one of the shortcomings of normal pipes: you can’t grab one end of a normal pipe that was created by an unrelated process.

If I run two individual copies of a program, they can both call pipe() all they want and still not be able to speak to one another.

Question

WHY CAN'T TWO SEPARATE PROCESSES COMMUNICATE WITH EACH OTHER VIA PIPE?

FIFO

This is because you must pipe(), then fork() to get a child process that can communicate to the parent via the pipe.)

With FIFOs, though, each unrelated process can simply open() the pipe and transfer data through it.

Since the FIFO is actually a file on disk, you have to do some fancy-schmancy stuff to create it. It's not that hard. You just have to call mkfifo() with the proper arguments.

```
mkfifo("myfifo", 0644);
```

*a FIFO can also be created from the command line using the Unix mkfifo command

Producers and Consumers

Once the FIFO has been created, a process can start up and open it for reading or writing using the standard open() system call.

Understanding the process becomes significantly clearer when examined through practical code examples. To illustrate interprocess communication using a FIFO, the following two programs are presented:

- speak.c: responsible for sending data through the FIFO.
- tick.c: reads and processes data received from the FIFO.

Code Example

```
#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

#include <fcntl.h>

#include <sys/stat.h>

#include <sys/types.h>

#define FIFO_PATH "/tmp/myfifo"

int main() {

    char message[] = "Hello, Child!";

    if (mkfifo(FIFO_PATH, 0666) == -1) {

        perror("mkfifo");

        exit(EXIT_FAILURE);

    }
```

```
    pid_t pid = fork();

    if (pid == -1) {
        perror("fork");
        exit(EXIT_FAILURE);
    }

    if (pid > 0) {  // Parent process
        int fd = open(FIFO_PATH, O_WRONLY);
        write(fd, message, sizeof(message));
        close(fd);

    } else {  // Child process
        int fd = open(FIFO_PATH, O_RDONLY);
        char buffer[100];
        read(fd, buffer, sizeof(buffer));
        printf("Child received: %s\n", buffer);
        close(fd);
    }

    unlink(FIFO_PATH); // Remove the FIFO

    return 0;
}
```

Header Files and Define: Similar to the previous example, these lines include necessary header files and define a macro FIFO_PATH which represents the path to the FIFO.
Main Function: Entry point of the program. It starts by defining a character array message containing the data to be sent through the FIFO.
FIFO Creation (mkfifo): mkfifo system call is used to create a FIFO with the specified path (FIFO_PATH). The permissions 0666 allow read and write access to everyone. If unsuccessful, it prints an error and exits.
Forking: fork is used to create a child process. If unsuccessful, it prints an error and exits.
Parent Process:
Opens the FIFO for writing (O_WRONLY).
Writes the message to the FIFO using write system call.
Closes the file descriptor associated with the FIFO.
Child Process:
Opens the FIFO for reading (O_RDONLY).
Reads data from the FIFO into the buffer using read system call.
Prints the received message.
Closes the file descriptor associated with the FIFO.
Cleanup and FIFO Removal: The program unlinks (removes) the FIFO using unlink after both processes have completed their communication.

Revision

pipe()

Redirecting process I/O

Agenda

Interprocess Communication

- FIFO
- Shared Memory

Attribution

TODAY'S LECTURE IS BASED ON BEEJ'S GUIDE TO INTERPROCESS COMMUNICATION

Question

WHAT IS THE LIMITATION OF USING PIPE?

FIFO

A FIFO (“First In, First Out”, pronounced “Fy-Foh”) is sometimes known as a named pipe.

It’s like a pipe, except that it has a name! I

- n this case, the name is that of a file that multiple processes can open() and read and write to.

This latter aspect of FIFOs is designed to let them get around one of the shortcomings of normal pipes: you can’t grab one end of a normal pipe that was created by an unrelated process.

If I run two individual copies of a program, they can both call pipe() all they want and still not be able to speak to one another.

Question

WHY CAN'T TWO SEPARATE PROCESSES COMMUNICATE WITH EACH OTHER VIA PIPE?

FIFO

This is because you must pipe(), then fork() to get a child process that can communicate to the parent via the pipe.)

With FIFOs, though, each unrelated process can simply open() the pipe and transfer data through it.

Since the FIFO is actually a file on disk, you have to do some fancy-schmancy stuff to create it. It's not that hard. You just have to call mkfifo() with the proper arguments.

```
mkfifo("myfifo", 0644);
```

*a FIFO can also be created from the command line using the Unix mkfifo command

Producers and Consumers

Once the FIFO has been created, a process can start up and open it for reading or writing using the standard open() system call.

Understanding the process becomes significantly clearer when examined through practical code examples. To illustrate interprocess communication using a FIFO, the following two programs are presented:

- speak.c: responsible for sending data through the FIFO.
- tick.c: reads and processes data received from the FIFO.

Summary

FIFO Creation (mkfifo): The program starts by creating a FIFO (named pipe) using the mkfifo system call. The FIFO path is specified as "/tmp/myfifo", and it is created with read and write permissions for everyone (0666).

Forking a Child Process: The program forks into two processes - the parent and the child.

Parent Process Actions:

The parent process opens the FIFO for writing (O_WRONLY).

It writes the contents of the message array to the FIFO using the write system call.

The parent process then closes the file descriptor associated with the FIFO.

Child Process Actions:

The child process opens the FIFO for reading (O_RDONLY).

It reads data from the FIFO into the buffer array using the read system call.

The child process prints the received message to the standard output.

The child process then closes the file descriptor associated with the FIFO.

Cleanup and FIFO Removal: After both processes have completed their communication, the program unlinks (removes) the FIFO using the unlink system call.

Another example

https://www.youtube.com/watch?v=NHs42-rJeWU

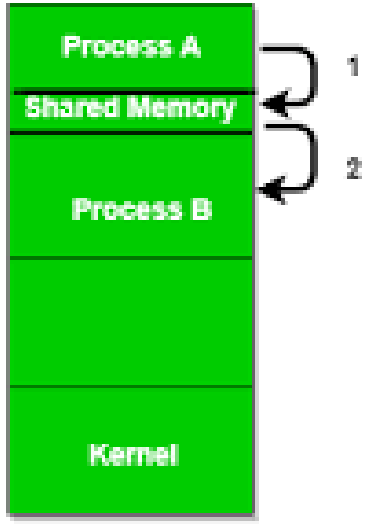
Efficient Communication with Shared Memory

Shared memory allows multiple processes to communicate by attaching the same block of memory to their address space.

Characteristics:

- Fast Local Communication
- Memory Model: Allocation, Attachment, Detachment

- Control and Deallocation:
  - Use of shmctl for managing shared memory segments



Working with Shared Memory

Key Steps:

- Allocation: shmget function
- Attachment and Detachment: shmat and shmdt functions
- Controlling and Deallocating: shmctl function

Note: Importance of synchronization and race condition prevention

Pros and Cons of Shared Memory

- Pros:
  - Fast bidirectional communication
  - Flexibility for reading and writing
- Cons:
  - Need for a protocol to prevent race conditions
  - Linux not guaranteeing exclusive access with IPC_PRIVATE

Shared Memory

Shared memory is a region of memory that can be accessed by multiple processes.

Fastest form of IPC (Interprocess Communication).

Requires synchronization to avoid race conditions.

Ideal for large data sharing.

Shared Memory - System Calls

shmget(key, size, shmflg):

- key: unique identifier (typically from ftok)
- size: size of shared memory segment in bytes
- shmflg: flags like IPC_CREAT, permissions

shmat(shmid, shmaddr, shmflg):

- shmid: ID returned by shmget
- shmaddr: desired attach address (usually NULL)
- shmflg: flags (0 or SHM_RDONLY)

Shared Memory - System Calls

shmdt(shmaddr):

- shmaddr: address returned by shmat

shmctl(shmid, cmd, buf):

- shmid: ID from shmget
- cmd: IPC_RMID (remove), IPC_STAT, etc.
- buf: pointer to struct shmid_ds (can be NULL for some cmds)

Shared Memory – Process Flow

1. Processes agree on a key using ftok()
2. One process creates the segment with shmget()
3. Both attach to memory using shmat()
4. Data is written/read via shared memory
5. Synchronization is used (e.g., semaphores)
6. Processes detach with shmdt() when done
7. One process deletes the segment using shmctl(..., IPC_RMID)

Shared Memory

Creating Shared Memory

```
key_t key = ftok("shmfile", 65);
int shmid = shmget(key, 1024, 0666|IPC_CREAT);
```

Attaching and Writing

```
char *str = (char*) shmat(shmid, (void*)0, 0);
printf("Write Data : ");
gets(str);
```

Shared Memory

Reading from Shared Memory

```
char *str = (char*) shmat(shmid, (void*)0, 0);
printf("Data read from memory: %s\n", str);
```

Detaching and Deleting

```
shmdt(str);
shmctl(shmid, IPC_RMID, NULL);
```

Example

```
#include <sys/ipc.h>
#include <sys/shm.h>
#include <stdio.h>

int main() {
    key_t key = ftok("shmfile",65);
    int shmid = shmget(key, 1024, 0666|IPC_CREAT);
    char *str = (char*) shmat(shmid, (void*)0, 0);
    printf("Write Data : ");
    gets(str);
    printf("Data written: %s\n", str);
    shmdt(str);
    return 0;
}
```

Example

```c
#include <sys/ipc.h>
#include <sys/shm.h>
#include <stdio.h>

int main() {
    key_t key = ftok("shmfile",65);
    int shmid = shmget(key,1024,0666);
    char *str = (char*) shmat(shmid, (void*)0, 0);
    printf("Data read: %s\n", str);
    shmdt(str);
    shmctl(shmid, IPC_RMID, NULL);
    return 0;
}
```

Code example

```
#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

#include <sys/shm.h>

#include <sys/wait.h>

#define SHM_SIZE 1024  // Size of the shared memory segment

int main() {

    int shmid;

    pid_t pid;

    char *shmaddr;

    // Create a shared memory segment

    shmid = shmget(IPC_PRIVATE, SHM_SIZE, IPC_CREAT | 0666);

    if (shmid == -1) {

        perror("shmget");

        exit(EXIT_FAILURE);    }

    // Fork a child process

    pid = fork();
```

```
if (pid == -1) {
    perror("fork");
    exit(EXIT_FAILURE);
}

if (pid > 0) {  // Parent process
    // Attach the shared memory segment
    shmaddr = shmat(shmid, NULL, 0);
    if (shmaddr == (char *)-1) {
        perror("shmat");
        exit(EXIT_FAILURE);
    }

    // Write data to shared memory
    sprintf(shmaddr, "Hello from Parent!");

    // Wait for the child to finish
    wait(NULL);

    // Detach the shared memory segment
    if (shmdt(shmaddr) == -1) {
        perror("shmdt");
        exit(EXIT_FAILURE);
    }

    // Remove the shared memory segment
    if (shmctl(shmid, IPC_RMID, NULL) == -1) {
        perror("shmctl");
        exit(EXIT_FAILURE);
    }
```

```
    } else {  // Child process
        // Attach the shared memory segment
        shmaddr = shmat(shmid, NULL, 0);
        if (shmaddr == (char *)-1) {
            perror("shmat");
            exit(EXIT_FAILURE);
        }

        // Read data from shared memory
        printf("Child received: %s\n", shmaddr);

        // Detach the shared memory segment
        if (shmdt(shmaddr) == -1) {
            perror("shmdt");
            exit(EXIT_FAILURE);
        }

        exit(EXIT_SUCCESS);
    }

    return 0;
}
```

Header Files: These lines include necessary header files. stdio.h for standard input/output, stdlib.h for standard library functions, unistd.h for fork, and sys/shm.h and sys/wait.h for shared memory and wait system calls.
Macro Definition: #define SHM_SIZE 1024 defines a macro named SHM_SIZE with the value 1024. This macro is used later to specify the size of the shared memory segment.
Main Function: The program's entry point is the main function.
Variable Declarations:
int shmid: Variable to store the shared memory identifier.
pid_t pid: Variable to store process ID (PID).
char *shmaddr: Variable to store the address of the shared memory.
Shared Memory Creation: shmget system call is used to create a new shared memory segment. IPC_PRIVATE is used as the key, SHM_SIZE specifies the size, and IPC_CREAT | 0666 sets permissions. If unsuccessful, it prints an error using perror and exits.
Forking: fork system call is used to create a child process. If the fork fails, it prints an error and exits.
Parent Process: Code block executed by the parent process.
Attach Shared Memory: shmat is used to attach the shared memory segment. If unsuccessful, it prints an error and exits.
Write Data and Wait: Parent writes a message to the shared memory and waits for the child process to finish using wait system call.
Child Process: Code block executed by the child process.
Attach Shared Memory: The child attaches to the same shared memory segment.
Child Process Actions: The child reads data from shared memory, prints it, detaches it from shared memory, and exits successfully.
Cleanup in Parent Process: The parent detaches from shared memory and removes the shared memory segment after the child has finished executing.

Another example

https://www.youtube.com/watch?v=Yb6pc_OU5x8

Question

WHAT CHALLENGES DO YOU SEE IF BOTH PROCESSES INTEND TO WRITE?

Need for Synchronization

Shared memory lacks built-in synchronization.

Use semaphores, mutexes, or condition variables.

Avoid race conditions and data corruption.

Question

WHAT IS ANOTHER CHALLENGE WITH SYNCHRONIZATION?

Kernel-managed semaphores

Semaphores are separate IPC objects (not part of shared memory)

Created via semget(), controlled via semctl(), semop()

Kernel-managed: stored in kernel space, not user space

Processes can share semaphores using a common key (like ftok())

Advantages/Disadvantages

Advantages

Very fast – no context switching

Efficient for large data

Flexible for many IPC patterns

Disadvantages

Requires manual synchronization

Debugging is complex

Must manage cleanup explicitly

Comparison Table

Feature	Pipe	FIFO	Shared Memory
Unrelated Processes	No	Yes	Yes
Speed	Moderate	Moderate	Fast
Required Sync	No	No	Yes
Data size	Limited	Limited	Large